Programación y pensamiento computacional

Nombre: _____ Fecha: _____

Qué vas a aprender

- Aplicar los cuatro pilares del pensamiento computacional a un problema
- Escribir algoritmos con secuencias, condiciones y bucles
- Crear y depurar programas sencillos por bloques o en texto
- Usar variables, funciones y listas para escribir programas ordenados
- Comparar algoritmos de búsqueda y ordenación
- Usar operadores, listas y bucles while y for en Python
- Comparar la eficiencia de algoritmos de búsqueda y ordenación

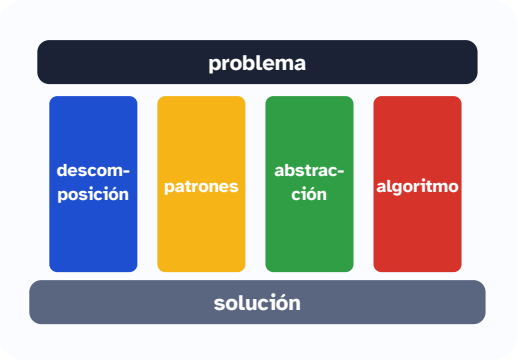
Para pensar

¿Cómo le explicarías a un robot que no sabe nada cómo preparar un bocadillo, si solo entiende órdenes muy simples?

Explicación

1.1 Los cuatro pilares

El **pensamiento computacional** es una forma de resolver problemas que se puede aplicar con o sin ordenador. Tiene cuatro pilares: **descomposición** (dividir un problema grande en partes pequeñas), **reconocimiento de patrones** (ver qué se repite o se parece a algo ya resuelto), **abstracción** (quedarse con lo importante e ignorar los detalles) y algoritmo (escribir los pasos ordenados para resolverlo).



1.2 Algoritmos y diagramas de flujo

Un **algoritmo** es una secuencia finita y ordenada de instrucciones que resuelve un problema. Se puede escribir en lenguaje natural, en **pseudocódigo** o con un diagrama de flujo, donde los óvalos marcan el inicio y el fin, los rectángulos las acciones, los rombos las decisiones (sí/no) y los paralelogramos la entrada y salida de datos. Un buen algoritmo es **preciso** (sin ambigüedades), **finito** (termina) y da siempre el mismo resultado con los mismos datos.



1.3 Las estructuras básicas

Cualquier programa se construye combinando tres estructuras. La **secuencia**: instrucciones una detrás de otra. La **selección** o **condicional**: si se cumple algo, haz esto; si no, lo otro. Y la **repetición** o **bucle**: repetir unas instrucciones un número de veces o mientras se cumpla una condición. Además se usan **variables**, cajas con nombre que guardan datos (puntos, vidas, nombre del jugador), y **operadores** para comparar y calcular.



2.1 Variables y tipos de datos

Una **variable** es un espacio de memoria con nombre donde el programa guarda un dato que puede cambiar: los puntos de un juego, la edad de un usuario, un nombre. Los datos tienen un **tipo**: **enteros** (7), **decimales** (3.5), **textos** o cadenas ("Hola"), **booleanos** (verdadero o falso) y **listas** que agrupan varios valores ([4, 8, 15]). Con ellos se hacen operaciones aritméticas (+, –, *, /), comparaciones (==, <, >) y operaciones lógicas (y, o, no). Elegir nombres claros para las variables, como `puntos` en vez de `x`, hace el programa mucho más fácil de entender.

Tipo	Ejemplo	Uso
Entero	15	Contar
Decimal	9.75	Medir
Texto	"Ana"	Nombres, mensajes
Booleano	True / False	Condiciones
Lista	[3, 7, 2]	Grupos de datos

Tipos de datos

Tipo	Nombre	Ejemplo
Entero	int	42
Decimal	float	3.14
Texto	str	"hola"
Booleano	bool	True
Lista	list	[1, 2, 3]

2.2 Operadores y expresiones

Una **expresión** combina datos, variables y **operadores** para obtener un valor. Los operadores **aritméticos** de Python son `+`, `-`, `*`, `/` (división con decimales), `//` (división entera), `%` (resto o módulo) y `**` (potencia). El resto es muy útil: `n % 2 == 0` indica si un número es par. Como en matemáticas, primero se hacen las potencias, luego multiplicaciones y divisiones y al final sumas y restas; los paréntesis cambian el orden.

Los operadores de **comparación** (`==`, `!=`, `<`, `>`, `<=`, `>=`) dan como resultado un booleano: `True` o `False`. Los operadores **lógicos** combinan condiciones: `and` exige que se cumplan las dos (edad >= 12 and edad <= 16), `or` basta con una y `not` invierte el resultado. Son las mismas puertas lógicas AND, OR y NOT de la electrónica digital. Un error muy frecuente es confundir `=`, que **asigna** un valor a una variable, con `==`, que **compara** dos valores. Otro es sumar texto y números: `"5" + 3` da error, porque primero hay que convertir con `int("5")`.

Operadores

Operador	Ejemplo	Da
//	17 // 5	3
%	17 % 5	2
******	2 ** 5	32
==	3 == 4	False
and	True and False	False

2.3 Listas y cadenas de texto

Cuando hay muchos datos del mismo tipo no se crea una variable para cada uno, sino una **lista**: `notas = [7, 5, 9, 4]`. Cada elemento tiene una posición o **índice**, que en Python empieza en `0`: `notas[0]` es 7 y `notas[3]` es 4. Con

`len(notas)` se sabe cuántos elementos tiene, con `notas.append(8)` se añade uno al final y con `notas.remove(5)` se quita. Las listas se recorren con un bucle: `for n in notas:` repite las instrucciones una vez por cada elemento.

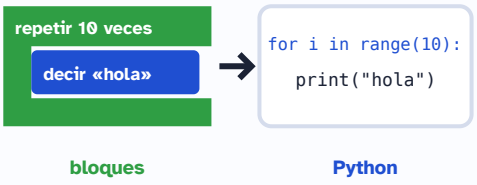
Las **cadenas** de texto funcionan de forma parecida: `nombre = "Python"` tiene 6 caracteres y `nombre[0]` es la «P». Se pueden **trocear** (`nombre[0:3]` da "Pyt"), pasar a mayúsculas con `.upper()` o unir con +. Las funciones `sum()`, `max()` y `min()` trabajan directamente con listas de números, y `sorted()` devuelve la lista ordenada. Las listas son la base de casi todos los programas reales: la lista de canciones de una aplicación, los mensajes de un chat o los puntos de un mapa.

notas = [7, 5, 9, 4]

Índice	0	1	2	3
Valor	7	5	9	4
Acceso	notas[0]	notas[1]	notas[2]	notas[3]

3.1 De los bloques al código

Para empezar se usan lenguajes **por bloques**, como Scratch o MakeCode, donde las instrucciones encajan como piezas de puzle y no hay errores de escritura. Después se pasa a lenguajes **de texto**, como Python o JavaScript, más potentes. La lógica es la misma: el bloque «repetir 10 veces» se escribe en Python `for i in range(10):`. Los programas pueden responder a **eventos** (al pulsar una tecla, al tocar un objeto), lo que permite crear juegos, animaciones y aplicaciones.



misma lógica, distinta escritura

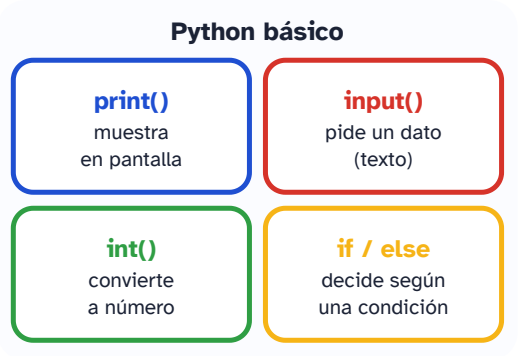
3.2 Programar con Scratch

Scratch, creado en el MIT, permite programar encajando bloques de colores, sin errores de escritura. Cada **objeto** (sprite) tiene sus programas, que empiezan con un **evento** («al hacer clic en la bandera»). Los bloques se agrupan por categorías: movimiento, apariencia, sonido, control (bucles y condiciones), sensores, operadores y variables. Con ellos se crean juegos, animaciones e historias interactivas. Los objetos pueden comunicarse enviando **mensajes**. Es ideal para aprender la lógica, porque las estructuras son las mismas que en cualquier lenguaje: secuencia, condición y bucle.



3.3 Primeros pasos en Python

Python es uno de los lenguajes más usados del mundo, en ciencia, inteligencia artificial o páginas web, y su sintaxis es muy legible. `print("Hola")` muestra un texto; `nombre = input("¿Cómo te llamas? ")` pide un dato; `if nota >= 5:` abre una condición, y `for i in range(10):` repite diez veces. En Python la sangría (los espacios al principio de la línea) indica qué instrucciones están dentro de un bloque, así que es obligatoria. Los textos que llegan con `input` hay que convertirlos con `int()` si se quiere operar con ellos.



3.4 Bucles en Python: while y for

Python tiene dos bucles. El bucle **while** («mientras») repite mientras se cumpla una condición; se usa cuando no sabemos cuántas vueltas habrá, por ejemplo, pedir una contraseña hasta que sea correcta. Dentro del bucle algo debe cambiar para que la condición deje de cumplirse; si no, se produce un bucle infinito y el programa no termina nunca.

El bucle **for** recorre una secuencia: una lista, una cadena o un rango de números. `range(5)` genera 0, 1, 2, 3, 4; `range(1, 11)` va del 1 al 10, y `range(0, 20, 5)` salta de 5 en 5. Dos patrones aparecen una y otra vez. El **contador** suma 1 cada vez que pasa algo (`aprobados = aprobados + 1`). El **acumulador** va sumando valores (`total = total + nota`); al final, la media es `total / len(notas)`. La orden `break` sale del bucle antes de tiempo, útil cuando ya se ha encontrado lo que se buscaba. Saber elegir el bucle adecuado y comprobar que siempre termina es una de las claves de la programación.



3.5 Funciones y modularidad

Una **función** es un trozo de programa con nombre que realiza una tarea concreta y se puede usar muchas veces. Recibe datos de entrada (**parámetros**) y puede devolver un resultado. Por ejemplo, una función `area_rectangulo(base, altura)` devuelve base · altura. Dividir un programa en funciones es aplicar la **descomposición**: cada parte es más fácil de escribir, probar y corregir, y se puede reutilizar en otros programas. En Scratch, las funciones se crean con «Mis bloques».



3.6 Programación por eventos

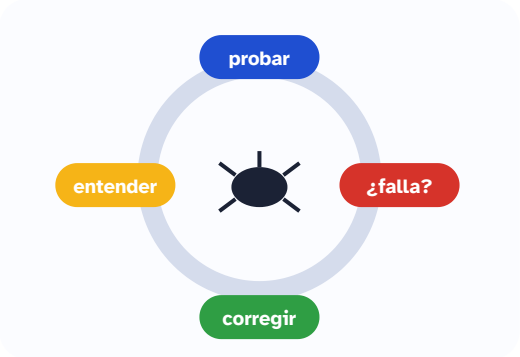
En los programas que hemos visto, las instrucciones se ejecutan en orden de principio a fin. Pero un videojuego, una app del móvil o una página web esperan a que ocurra algo: un clic, una tecla, un mensaje que llega o un sensor que cambia. Esto es la **programación orientada a eventos**. El programador escribe **manejadores**: trozos de código que se ejecutan cuando ocurre un evento concreto. En Scratch son los bloques amarillos «al hacer clic en este objeto» o «al presionar la tecla espacio»; en App Inventor, «cuando Botón1.Clic»; en JavaScript, `boton.onclick`.

Mientras no pasa nada, el programa espera en un **bucle de eventos** sin gastar apenas recursos. Varios objetos pueden comunicarse enviando **mensajes** (el bloque «enviar mensaje» de Scratch): cuando el jugador toca la meta, se envía «fin» y el escenario muestra la pantalla de victoria. Este modelo hace que las aplicaciones respondan al usuario de forma natural. Su dificultad es que el orden en que se ejecutan las cosas depende de lo que haga el usuario, así que hay que probar muchas combinaciones distintas.



4.1 Probar y depurar

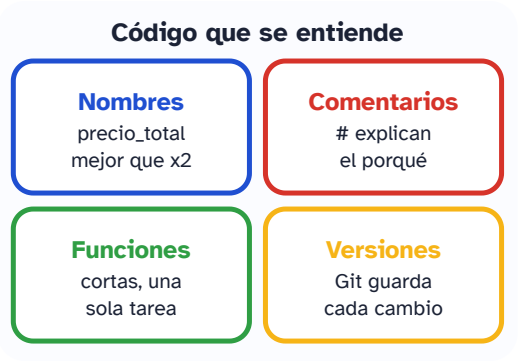
Casi ningún programa funciona a la primera. Los fallos se llaman **bugs**, y buscarlos y corregirlos es **depurar**. Hay errores de **sintaxis** (algo mal escrito, que impide ejecutar) y errores de **lógica** (el programa funciona pero hace otra cosa). Para depurar: lee el mensaje de error, prueba con datos sencillos cuyo resultado conoces, muestra el valor de las variables paso a paso y cambia una sola cosa cada vez. Explicar el código en voz alta (a un compañero o a un patito de goma) ayuda mucho.



4.2 Buenas prácticas de programación

Un programa no solo lo lee el ordenador: también lo leerán tus compañeros y tú mismo dentro de un mes. Por eso conviene seguir unas **buenas prácticas**. Usa **nombres descriptivos**: `precio_total` se entiende; `x2`, no. Escribe **comentarios** (en Python empiezan con #) para explicar el *porqué* de las partes difíciles, no lo que es evidente. Mantén las funciones cortas, cada una con una sola tarea, y evita repetir código: si copias y pegas tres veces lo mismo, es momento de crear una función.

Antes de programar, escribe el algoritmo en **pseudocódigo** o dibuja el diagrama de flujo. Prueba el programa con **casos límite**: la lista vacía, el número 0, una nota de exactamente 5, un texto con tildes. Los programadores profesionales guardan cada cambio con un **sistema de control de versiones** como **Git**, que permite volver atrás y trabajar en equipo sin pisarse; plataformas como GitHub alojan millones de proyectos de código abierto. Por último, ten en cuenta la **accesibilidad** y la **ética**: un programa debe poder usarlo todo el mundo y no debe recoger más datos de los necesarios.



4.3 Buscar y ordenar

Dos problemas clásicos muestran que hay algoritmos mejores que otros. Para **buscar** un dato en una lista desordenada solo cabe mirar uno por uno (**búsqueda lineal**). Pero si la lista está ordenada, la **búsqueda binaria** mira el elemento central y descarta la mitad en cada paso: en una lista de 1.000 elementos encuentra el dato en unos 10 pasos en vez de hasta 1.000. Para **ordenar**, el método de la burbuja compara parejas vecinas y las intercambia si están desordenadas; es sencillo pero lento, y por eso se usan métodos más eficientes con muchos datos.



4.4 La eficiencia de los algoritmos

Dos algoritmos pueden resolver el mismo problema y tardar tiempos muy distintos. Para compararlos no se miden segundos, que dependen del ordenador, sino el **número de pasos** en función del tamaño de los datos, *n*. La búsqueda lineal necesita, en el peor caso, **n pasos**: si la lista se duplica, el tiempo se duplica. La búsqueda binaria solo necesita unos $\log_2 n$ pasos: con 1.000 elementos, unos 10; con un millón, unos 20. El método de la burbuja para ordenar hace unas n^2 comparaciones: con 1.000 elementos, un millón de comparaciones.

Por eso los informáticos buscan algoritmos más listos, como el **ordenamiento por mezcla** (*merge sort*), que divide la lista en mitades, las ordena y las une, con unos $n \cdot \log_2 n$ pasos. Con datos pequeños la diferencia apenas se nota, pero los buscadores, las redes sociales o los mapas trabajan con miles de millones de datos: un algoritmo ineficiente haría que una búsqueda tardara horas en lugar de milisegundos. También importa la **memoria** que usa el algoritmo. Elegir bien el algoritmo suele mejorar más un programa que comprar un ordenador más potente.



Pensamiento computacional en un problema real: organizar una excursión

Descomponer	Patrones	Abstraer	Algoritmo
transporte, comida, permisos, horario	la excursión del año pasado sirve de base	importa nº de alumnos, no su color favorito	lista de pasos con fechas y avisos

Ejercicios

1. **Indica el tipo de dato: 42, "42", 3.14, True, [1, 2, 3].**

2. **¿Qué muestra este programa? a = 5; b = 3; a = a + b; print(a * 2).**

3. **Escribe un bucle en Python que muestre los números del 1 al 10.**

4. **Escribe un programa que pida la edad y diga «Puedes votar» si es 18 o más.**

5. **Escribe una función que reciba un número y devuelva su doble.**

6. **Describe un algoritmo para encontrar el número mayor de una lista.**

7. **En Scratch, ¿qué bloque usarías para que un objeto se mueva sin parar?**

8. **Encuentra el error: if nota > 5: print("Aprobado") (un alumno con un 5 aparece como suspenso).**

9. **Ordena con el método de la burbuja la lista [5, 2, 4, 1] e indica la primera pasada.**

10. **¿Por qué conviene dividir un programa en funciones?**

11. **Aplica los cuatro pilares al problema de organizar una excursión.**

12. **¿Qué valen 17 // 3, 17 % 3 y 2 ** 4 en Python?**

13. **Dada la lista colores = ["rojo", "verde", "azul"], ¿qué es colores[1]? ¿Y len(colores)?**

14. **¿Qué diferencia hay entre = y == en Python?**

15. **Escribe qué números genera range(2, 12, 3).**

16. **Pon dos ejemplos de eventos en una aplicación del móvil.**

17. **Escribe un comentario útil para la línea total = total * 1.21.**

18. **Escribe en Python una condición que sea True si un número n es par y mayor que 10.**

19. **Escribe un programa que pida números hasta que el usuario escriba 0 y muestre la suma.**

20. **Recorre la lista notas = [6, 3, 8, 5, 4] y cuenta cuántos aprobados hay.**

21. **Calcula la media de una lista de notas sin usar sum().**

22. **Encuentra el fallo: i = 1; while i <= 5: print(i). ¿Qué pasa y cómo se arregla?**

23. **Escribe una función es_mayor(edad) que devuelva True si la edad es 18 o más.**

24. **¿Qué muestra: palabra = "robotica"; print(palabra[0:4].upper())?**

25. **Describe en Scratch un juego en el que, al tocar una moneda, se suma un punto y la moneda aparece en otro sitio.**

26. **¿Cuántos pasos necesita como máximo la búsqueda binaria en una lista ordenada de 1.024 elementos? ¿Y la lineal?**

27. **Reescribe con nombres claros y un comentario: a = int(input()); b = a * 0.8; print(b).**

28. **Escribe una función mayor(lista) que devuelva el número más grande sin usar max().**

29. **Escribe un programa que pida una contraseña y dé solo 3 intentos.**

30. **Escribe un programa que muestre la tabla de multiplicar de un número pedido al usuario.**

31. **Una lista de 1.000 elementos se ordena con burbuja y otra con mezcla. Estima las comparaciones de cada una y explica la diferencia con 1.000.000 de elementos.**

32. **Escribe una función que reciba una palabra y diga si es un palíndromo (se lee igual al revés).**

33. **Diseña los casos de prueba para una función que dice si una nota es «Suspenso», «Aprobado», «Notable» o «Sobresaliente».**

34. **Explica por qué en un videojuego por eventos pueden aparecer errores que no salen en las pruebas normales.**

35. **Escribe un programa que cuente cuántas vocales tiene una frase.**

Practica

1. **Dividir un problema grande en partes pequeñas es...**

   A) abstracción
   B) reconocer patrones
   C) descomposición
   D) depuración

2. **En un diagrama de flujo, una decisión se dibuja con...**

   A) un hexágono
   B) un óvalo
   C) un rombo
   D) un rectángulo

3. **¿Qué estructura repite instrucciones?**

   A) El bucle
   B) La secuencia
   C) La variable
   D) La función

4. **¿Qué es una variable?**

   A) Un error del programa
   B) Un espacio con nombre que guarda un dato
   C) Una instrucción de salida
   D) Un tipo de bucle

5. **Un programa se ejecuta pero suma en vez de restar. Es un error de...**

   A) sintaxis
   B) red
   C) lógica
   D) hardware

6. **En Python, la sangría...**

   A) hace el programa más rápido
   B) es opcional
   C) indica los bloques de código
   D) solo sirve para que quede bonito

7. **True y False son datos de tipo...**

A) texto
B) booleano
C) entero
D) decimal

8. **== sirve para...**

A) asignar
B) comparar
C) multiplicar
D) sumar

9. **La búsqueda binaria necesita que la lista esté...**

A) ordenada
B) desordenada
C) vacía
D) en mayúsculas

10. **Una función que recibe datos los recibe como...**

A) variables globales
B) parámetros
C) eventos
D) sprites

11. **¿Qué da 17 % 5 en Python?**

A) 3
B) 2
C) 3.4
D) 85

12. **En la lista [4, 8, 15], ¿qué índice tiene el 15?**

A) 0
B) 1
C) 2
D) 3

13. **¿Qué bucle usarías para pedir una contraseña hasta que sea correcta?**

A) for con range(10)
B) while
C) Ninguno

D) if

14. **¿Qué valor da True and False?**

    A) True

    B) False

    C) Error

    D) 1

15. **Una variable que va sumando valores en un bucle es...**

    A) un contador

    B) un acumulador

    C) un evento

    D) un índice

16. **«Al hacer clic en este objeto» es un bloque de...**

    A) bucle

    B) evento

    C) operador

    D) lista

17. **¿Qué hace break dentro de un bucle?**

    A) Repite el bucle

    B) Sale del bucle

    C) Borra la variable

    D) Detiene el ordenador

18. **Con una lista ordenada de un millón de elementos, la búsqueda binaria necesita unos...**

    A) 20 pasos

    B) 1.000 pasos

    C) 500.000 pasos

    D) un millón de pasos

19. **Git es...**

    A) un lenguaje de programación

    B) un sistema de control de versiones

    C) un tipo de bucle

    D) un editor de imagen

20. **¿Qué da "5" + 3 en Python?**

A) 8

B) 53

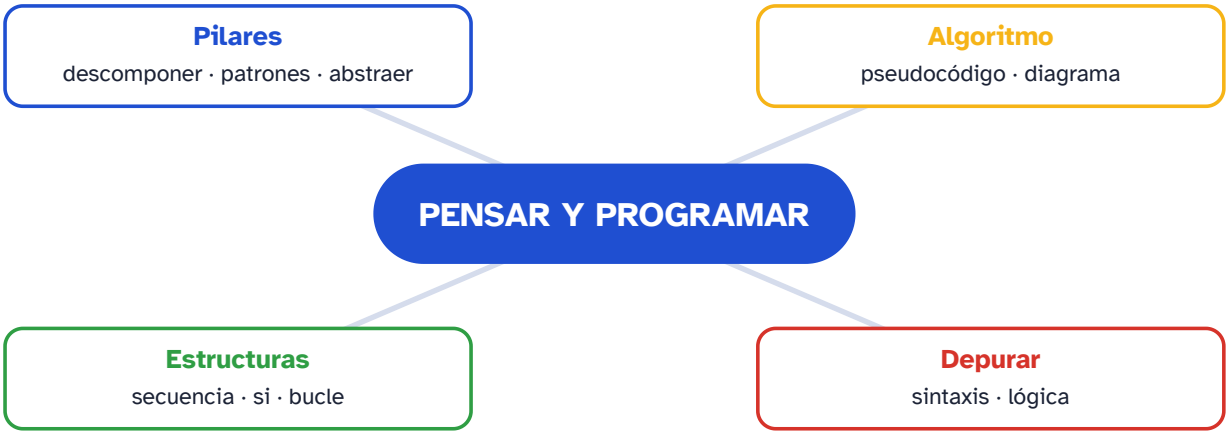
C) Un error

D) "8"

Explícalo con tus palabras

Incluye: Los cuatro pilares del pensamiento computacional con un ejemplo; Qué es un algoritmo y cómo se dibuja un diagrama de flujo; Secuencia, selección y bucle; Cómo pasar de un programa por bloques a Python; Errores de sintaxis y de lógica, y cómo depurarlos.

Resumen



Glosario

Pensamiento computacional	Forma de resolver problemas con descomposición, patrones, abstracción y algoritmos.
Algoritmo	Secuencia finita y ordenada de instrucciones que resuelve un problema.
Pseudocódigo	Algoritmo escrito en lenguaje parecido al natural, sin reglas de un lenguaje concreto.
Variable	Espacio de memoria con nombre que guarda un dato.
Booleano	Tipo de dato que solo puede valer True o False.
Operador lógico	and, or y not: combinan o invierten condiciones.
Lista	Colección ordenada de datos a la que se accede por su índice.
Índice	Posición de un elemento en una lista o cadena; empieza en `0`.
Bucle	Estructura que repite instrucciones.
Contador	Variable que aumenta en 1 cada vez que ocurre algo.
Acumulador	Variable que va sumando valores dentro de un bucle.

Función	Bloque de código con nombre que realiza una tarea y puede devolver un resultado.
Parámetro	Dato que recibe una función.
Evento	Algo que ocurre (un clic, una tecla) y que pone en marcha un trozo de código.
Depurar	Buscar y corregir los errores de un programa.
Control de versiones	Sistema que guarda el historial de cambios del código (Git).
Eficiencia	Cantidad de pasos o de memoria que necesita un algoritmo.

Tarjetas de repaso

Recórtalas: pregunta delante, respuesta detrás.

Los cuatro pilares

Descomposición, patrones, abstracción y algoritmo.

¿Qué es un algoritmo?

Una secuencia finita y ordenada de instrucciones que resuelve un problema.

Las tres estructuras básicas

Secuencia, selección (condicional) y repetición (bucle).

¿Qué es depurar?

Buscar y corregir los errores (bugs) de un programa.

Error de sintaxis

Algo mal escrito que impide que el programa se ejecute.

Variable

Espacio de memoria con nombre que guarda un dato que puede cambiar.

Función

Trozo de programa con nombre que hace una tarea y se puede reutilizar.

Búsqueda binaria

Busca en una lista ordenada descartando la mitad en cada paso.

Operador %

Da el resto de la división: n % 2 == 0 indica que n es par.

Índices de una lista

Empiezan en 0: lista[0] es el primer elemento.

while frente a for

while repite mientras se cumpla algo; for recorre una secuencia.

Eficiencia

Se mide en pasos según el tamaño n: lineal n, binaria $\log_2 n$, burbuja n^2.

Soluciones

1. **C)** Cada parte es más fácil de resolver.
2. **C)** El rombo tiene dos salidas: sí y no.
3. **A)** Por ejemplo, «repetir 10 veces» o «mientras…».
4. **B)** Su valor puede cambiar durante el programa.
5. **C)** Está bien escrito, pero hace otra cosa.
6. **C)** Es obligatoria.
7. **B)** Verdadero o falso.
8. **B)** = asigna; == compara.
9. **A)** Descarta la mitad en cada paso.
10. **B)** Por ejemplo, base y altura.
11. **B)** % calcula el resto de la división: $17 = 5 \cdot 3 + 2$.
12. **C)** Los índices empiezan en 0.
13. **B)** No se sabe cuántas veces habrá que repetir.
14. **B)** and exige que las dos condiciones sean ciertas.
15. **B)** El contador suma 1; el acumulador suma valores.
16. **B)** Se ejecuta cuando ocurre el clic.
17. **B)** Termina el bucle antes de tiempo.
18. **A)** $\log_2 1.000.000 \approx 20$.
19. **B)** Guarda el historial de cambios del código.
20. **C)** No se puede sumar texto y número sin convertir antes.

Soluciones de los ejercicios

1. Entero, texto, decimal, booleano y lista.
2. a vale 8, así que muestra 16.
3. for i in range(1, 11): print(i).
4. edad = int(input("Edad: ")); if edad >= 18: print("Puedes votar").
5. def doble(n): return n * 2.
6. Guarda el primero como mayor; recorre la lista y, si un número es mayor que el guardado, sustitúyelo; al final, el guardado es el mayor.
7. Un bloque «por siempre» con «mover 10 pasos» y «rebotar si toca un borde» dentro.
8. La condición debe ser nota >= 5, para incluir el 5.
9. Primera pasada: [2, 5, 4, 1] → [2, 4, 5, 1] → [2, 4, 1, 5]. Al final: [1, 2, 4, 5].
10. Porque cada parte es más fácil de escribir, probar y corregir, y se puede reutilizar.
11. Descomposición: transporte, comida, permisos, horario. Patrones: es parecida a otras excursiones del curso pasado. Abstracción: lo importante es el número de alumnos, el destino y el presupuesto (no el color del autobús). Algoritmo: la lista ordenada de tareas con sus fechas.
12. 17 // 3 = 5 (división entera), 17 % 3 = 2 (resto) y 2 ** 4 = 16 (potencia).
13. colores[1] es "verde", porque los índices empiezan en 0. len(colores) es 3.
14. = asigna un valor a una variable (x = 5). == compara dos valores y da True o False (x == 5).
15. 2, 5, 8, 11. Empieza en 2, salta de 3 en 3 y se detiene antes de 12.
16. Pulsar un botón, deslizar el dedo, recibir una notificación, girar el móvil o que cambie la batería.
17. # Añadimos el 21 % de IVA al total
18. n % 2 == 0 and n > 10

19. ```
    total = 0
    n = int(input("Número: "))
    while n != 0:
        total = total + n
        n = int(input("Número: "))
    print(total)
    ```

20. ```
    aprobados = 0
    for n in notas:
        if n >= 5:
            aprobados = aprobados + 1
    print(aprobados)  # muestra 3
    ```

21. ```
    total = 0
    for n in notas:
        total = total + n
    ```
    ```

```
media = total / len(notas)
print(media)
```

22. i nunca cambia, así que la condición siempre se cumple: bucle infinito que muestra 1 sin parar. Se arregla añadiendo i = i + 1 dentro del bucle.

23. ```
    def es_mayor(edad):
        return edad >= 18
    ```

24. Toma los caracteres de las posiciones 0 a 3, "robo", y los pasa a mayúsculas: ROBO.
25. Variable «puntos» a 0 al pulsar la bandera. En la moneda: por siempre, si toca al personaje → cambiar puntos en 1, ir a una posición aleatoria. Es un evento (tocar) dentro de un bucle.
26. Binaria: $\log_2 1.024 = 10$ pasos (cada paso divide la lista a la mitad). Lineal: hasta 1.024 pasos.

27. ```
    precio = int(input("Precio: "))
    # Aplicamos un descuento del 20 %
    precio_final = precio * 0.8
    print(precio_final)
    ```

28. ```
    def mayor(lista):
        m = lista[0]
        for x in lista:
            if x > m:
                m = x
        return m
    ```

29. ```
    intentos = 0
    while intentos < 3:
        clave = input("Clave: ")
        if clave == "tecno24":
            print("Acceso concedido")
            break
        intentos = intentos + 1
    if intentos == 3:
        print("Bloqueado")
    ```

30. ```
    n = int(input("Número: "))
    for i in range(1, 11):
        print(n, "x", i, "=", n * i)
    ```

31. Burbuja: unos $n^2 = 1.000.000$; mezcla: unos $n \cdot \log_2 n \approx 1.000 \cdot 10 = 10.000$. Con un millón: burbuja 10^{12} (un billón) y mezcla unos 20 millones. La diferencia crece muchísimo con el tamaño.
    ```

32. ```
    def palindromo(p):
        p = p.lower()
        return p == p[::-1]

    print(palindromo("Ana"))  # True
    ```
33. Probar valores normales (3, 6, 7.5, 9.5) y sobre todo los límites: 4.99, 5, 6.99, 7, 8.99, 9 y 10; además valores no válidos como -1, 11 o un texto, para comprobar que el programa avisa del error.
34. El orden de ejecución depende de lo que haga el jugador: dos eventos pueden ocurrir casi a la vez (tocar la meta y perder la última vida) y el resultado depende de cuál se procese primero. Hay que probar combinaciones raras y controlar el estado del juego.
35. ```
    frase = input("Frase: ").lower()
    vocales = 0
    for letra in frase:
        if letra in "aeiouáéíóú":
            vocales = vocales + 1
    print(vocales)
    ```